

Guía Esencial de C++: De Fundamentos a Semántica de Movimiento

Manual de referencia rápido con explicaciones concisas, conceptos clave y ejemplos prácticos de código (C++11 / C++17 / C++20)

48 CONCEPTOS CLAVE • EJEMPLOS COMPILABLES • ESTÁNDARES C++ MODERNOS

01. Tipos Fundamentales

FUNDAMENTOS

Los tipos fundamentales en C++ son los datos primarios integrados en el lenguaje. Incluyen enteros (`int`, `short`, `long`), coma flotante (`float`, `double`), caracteres (`char`, `wchar_t`), booleanos (`bool`) y el tipo especial `void`. Representan directamente conceptos del hardware y sus tamaños pueden variar según la arquitectura objetivo (32/64 bits).

```
#include <iostream>

int main() {
    int edad = 20;
    double altura = 1.75;
    char inicial = 'H';
    bool estudiante = true;

    std::cout << "Edad: " << edad << " | Bytes: " << sizeof(edad) << "\n";
    std::cout << "Altura: " << altura << " | Bytes: " << sizeof(altura) << "\n";
    return 0;
}
```

02. Variables y Scope

FUNDAMENTOS

Una variable es un contenedor nombrado que almacena un dato en memoria. El **scope** (ámbito) define la región del código donde la variable es visible y accesible. C++ utiliza ámbito de bloque delimitado por llaves `{}`: una variable creada dentro de un bloque nace al declararse y se destruye automáticamente al salir del bloque.

```
#include <iostream>

int global = 100; // Visible en todo el archivo

int main() {
    int local = 10; // Visible en main()
    {
        int bloque = 50; // Visible solo dentro de este {}
        std::cout << "Suma: " << (local + bloque) << "\n";
    }
    // 'bloque' ya no existe aquí.
    std::cout << "Global: " << global << "\n";
    return 0;
}
```

03. Operadores

FUNDAMENTOS

Símbolos que realizan operaciones matemáticas, lógicas o de manipulación de datos. Se dividen en: Aritméticos (`+` `-` `*` `/` `%`), Relacionales (`==` `!=` `<` `>` `<=` `>=`), Lógicos (`&&` `||` `!`), Asignación (`=` `+=` `-=` `*=`) y De Nivel de Bits (`&` `|` `^` `~` `<<` `>>`).

```
#include <iostream>

int main() {
    int a = 12, b = 5;
    bool es_par = (a % 2 == 0);
    bool rango = (a > 10) && (b < 10);
    a += 3; // Equivalente a a = a + 3

    std::cout << "a: " << a << " | Es par: " << es_par << " | Rango: " << rango << "\n";
    return 0;
}
```

04. if / switch / loops

FUNDAMENTOS

Estructuras de control de flujo. `if/else` toma decisiones basadas en expresiones booleanas. `switch` evalúa una expresión contra múltiples casos constantes. Los bucles (`for`, `while`, `do-while`) repiten bloques de código iterativamente.

```
#include <iostream>

int main() {
    int opcion = 1;
    if (opcion == 1) {
        std::cout << "Modo normal\n";
    }

    // Bucle for
    for (int i = 0; i < 3; ++i) {
        std::cout << "i = " << i << "\n";
    }
    return 0;
}
```

05. Funciones

FUNDAMENTOS

Bloques modulares de código encapsulado diseñados para realizar tareas específicas. Requieren definir un tipo de retorno, nombre, lista de parámetros y un cuerpo de ejecución. Ayudan a evitar duplicación y mejoran la legibilidad.

```
#include <iostream>

// Declaración
int calcular_cuadrado(int n);

int main() {
    std::cout << "Cuadrado de 4: " << calcular_cuadrado(4) << "\n";
    return 0;
}

// Definición
int calcular_cuadrado(int n) {
    return n * n;
}
```

06. Parámetros por Valor

FUNDAMENTOS

Modo de paso de argumentos por defecto en C++. La función recibe una **copia independiente** del valor suministrado. Las modificaciones dentro del cuerpo de la función no afectan a la variable original del invocador.

```
#include <iostream>

void intentar_modificar(int x) {
    x = 999; // Solo cambia la copia local
}

int main() {
    int original = 42;
    intentar_modificar(original);
    std::cout << "Original: " << original << "\n"; // Imprime 42
    return 0;
}
```

07. Referencias &

MEMORIA & PUNTEROS

Una referencia (**T&**) es un alias inmutable para una variable existente. No crea un nuevo objeto en memoria; operar sobre la referencia es idéntico a operar directamente sobre el objeto referenciado. Permite pasar argumentos a funciones sin realizar copias.

```
#include <iostream>

void duplicar(int& x) {
    x *= 2; // Modifica la variable original
}

int main() {
    int num = 15;
    duplicar(num);
    std::cout << "Num duplicado: " << num << "\n"; // Imprime 30
    return 0;
}
```

08. Punteros *

MEMORIA & PUNTEROS

Un puntero (`T*`) es una variable que almacena la **dirección de memoria** de otra variable. Se utiliza `&` para obtener la dirección y `*` para acceder (desreferenciar) al valor apuntado. A diferencia de las referencias, un puntero puede reasignarse o ser nulo (`nullptr`).

```
#include <iostream>

int main() {
    int val = 100;
    int* ptr = &val; // 'ptr' contiene la dirección de 'val'

    std::cout << "Dirección: " << ptr << "\n";
    std::cout << "Valor apuntado: " << *ptr << "\n";
    *ptr = 200; // Modifica 'val'
    std::cout << "Nuevo val: " << val << "\n";
    return 0;
}
```

09. const

MEMORIA & PUNTEROS

Especificador que indica inmutabilidad. Previene la modificación accidental de variables, miembros o retornos tras su inicialización. En funciones miembro de clases, `const` garantiza que el método no modificará el estado del objeto.

```
#include <iostream>

void mostrar(const std::string& msg) {
    std::cout << msg << "\n";
    // msg = "cambio"; // Error de compilación
}

int main() {
    const double TASA_IVA = 0.19;
    mostrar("Mensaje seguro");
    return 0;
}
```

10. Arrays (Arreglos estilo C)

STL & CONTENEDORES

Colección contigua de elementos del mismo tipo con un tamaño fijo en tiempo de compilación. No gestionan automáticamente su tamaño ni comprueban límites de acceso (Out-of-bounds). Al pasar a funciones, 'decaen' a un puntero a su primer elemento.

```
#include <iostream>

int main() {
    int numeros[4] = {10, 20, 30, 40};

    for (int i = 0; i < 4; ++i) {
        std::cout << "Index " << i << ": " << numeros[i] << "\n";
    }
    return 0;
}
```

11. std::array

STL & CONTENEDORES

Contenedor de la Librería Estándar (C++11) que envuelve un arreglo de tamaño fijo con semántica de clase. Se asigna en la pila (Stack), conoce su propio tamaño mediante `.size()`, y permite acceso seguro mediante `.at()`.

```
#include <iostream>
#include <array>

int main() {
    std::array<int, 3> datos = {1, 2, 3};

    std::cout << "Tamaño: " << datos.size() << "\n";
    std::cout << "Elemento 1: " << datos.at(1) << "\n";
    return 0;
}
```

12. std::vector

STL & CONTENEDORES

Arreglo dinámico de la STL. Almacena elementos de manera contigua en el Heap y redimensiona su capacidad automáticamente conforme se insertan (`push_back`) o eliminan datos. Es el contenedor secuencial por excelencia en C++.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec = {10, 20};
    vec.push_back(30); // Añade elemento al final

    for (int elem : vec) {
        std::cout << elem << " ";
    }
    std::cout << "\nTamaño: " << vec.size() << "\n";
    return 0;
}
```

13. std::string

STL & CONTENEDORES

Clase especializada de la STL para representar y manipular cadenas de texto variables. Administra automáticamente su memoria dinámica interna y proporciona métodos de búsqueda, concatenación y subcadenas.

```
#include <iostream>
#include <string>

int main() {
    std::string nombre = "Hugo";
    nombre += " Barrero"; // Concatenación
    std::cout << "Hola " << nombre << ", longitud: " << nombre.length() << "\n";
    return 0;
}
```

14. struct

POO & CLASES

Estructura de datos que permite agrupar múltiples variables relacionadas bajo un mismo tipo. En C++, la única diferencia técnica entre `struct` y `class` es que en `struct` todos los miembros son `public` por defecto.

```
#include <iostream>

struct Persona {
    std::string nombre;
    int edad;
};

int main() {
    Persona p1 = {"Hugo", 20};
    std::cout << p1.nombre << " tiene " << p1.edad << " años.\n";
    return 0;
}
```

15. class

POO & CLASES

Bloque fundamental de la Programación Orientada a Objetos en C++. Permite definir tipos con datos (atributos) y funciones (métodos). A diferencia de `struct`, en `class` los miembros son `private` por defecto.

```
#include <iostream>

class CuentaBancaria {
    double saldo; // Privado por defecto

public:
    void depositar(double monto) {
        if (monto > 0) saldo += monto;
    }
    double obtener_saldo() const { return saldo; }
};

int main() {
    CuentaBancaria cuenta;
    cuenta.depositar(100.0);
    std::cout << "Saldo: " << cuenta.obtener_saldo() << "\n";
    return 0;
}
```

16. public / private

POO & CLASES

Especificadores de acceso en clases. `public` otorga acceso desde cualquier parte del código. `private` restringe el acceso únicamente a los métodos dentro de la propia clase.

```
#include <iostream>

class Vehiculo {
private:
    int velocidad_maxima;

public:
    void set_velocidad(int v) { velocidad_maxima = v; }
    int get_velocidad() const { return velocidad_maxima; }
};

int main() {
    Vehiculo v;
    v.set_velocidad(180);
    std::cout << "Velocidad: " << v.get_velocidad() << " km/h\n";
    return 0;
}
```

17. Constructores y Destructores

POO & CLASES

Métodos especiales de ciclo de vida. El **Constructor** (`NombreClase()`) se invoca automáticamente al crear un objeto para inicializar sus datos. El **Destructor** (`~NombreClase()`) se ejecuta al destruir el objeto para liberar recursos.

```
#include <iostream>

class Recurso {
public:
    Recurso() { std::cout << "Recurso Creado\n"; }
    ~Recurso() { std::cout << "Recurso Destruído\n"; }
};

int main() {
    {
        Recurso r; // Llama al constructor
    } // Sale del scope -> Llama al destructor automáticamente
    return 0;
}
```

18. Encapsulamiento

POO & CLASES

Principio de la POO que oculta el estado interno de un objeto tras una interfaz bien definida. Evita modificaciones arbitrarias desde el exterior garantizando que los datos solo cambien mediante métodos autorizados.

```
#include <iostream>

class Termostato {
private:
    float temperatura = 20.0f;

public:
    void ajustar_temp(float t) {
        if (t >= -10.0f && t <= 50.0f) temperatura = t;
    }
    float obtener_temp() const { return temperatura; }
};

int main() {
    Termostato t;
    t.ajustar_temp(24.5f);
    std::cout << "Temp: " << t.obtener_temp() << " C\n";
    return 0;
}
```

19. Composición

POO & CLASES

Relación de diseño 'Tiene un' (Has-A) donde una clase contiene una o más instancias de otras clases como miembros. Permite construir tipos complejos reutilizando componentes independientes.

```
#include <iostream>

class Motor {
public:
    void arrancar() { std::cout << "Motor encendido\n"; }
};

class Auto {
private:
    Motor motor; // Composición: Auto tiene un Motor
public:
    void encender() { motor.arrancar(); }
};

int main() {
    Auto mi_auto;
    mi_auto.encender();
    return 0;
}
```


20. Herencia y Virtual

POO & CLASES

La herencia permite crear clases derivadas a partir de una clase base ('Es un'). La palabra clave `virtual` permite el polimorfismo dinámico: redefine funciones en clases derivadas para ejecutarlas dinámicamente mediante punteros o referencias a la base.

```
#include <iostream>

class Animal {
public:
    virtual void hacer_sonido() const { std::cout << "Sonido genérico\n"; }
};

class Perro : public Animal {
public:
    void hacer_sonido() const override { std::cout << "¡Guau!\n"; }
};

int main() {
    Animal* a = new Perro();
    a->hacer_sonido(); // Llama a Perro::hacer_sonido()
    delete a;
    return 0;
}
```

21. enum y enum class

ESTRUCTURA & GENERICOS

Las enumeraciones definen un conjunto de constantes enteras nombradas. Los `enum` tradicionales exportan sus nombres al scope circundante y permiten conversión implícita a entero. Los `enum class` (C++11) son fuertemente tipados y exigen calificación de ámbito (`Estado::Activo`).

```
#include <iostream>

enum class Estado { Inactivo, Activo, Pendiente };

int main() {
    Estado e = Estado::Activo;
    if (e == Estado::Activo) {
        std::cout << "Estado activo verificado.\n";
    }
    return 0;
}
```

22. namespaces

ESTRUCTURA & GENERICOS

Mecanismo para agrupar identificadores (clases, funciones, variables) bajo un espacio con nombre propio. Previene colisiones de nombres cuando se integran múltiples librerías.

```
#include <iostream>

namespace MiProyecto {
    void log(const std::string& msg) {
        std::cout << "[LOG]: " << msg << "\n";
    }
}

int main() {
    MiProyecto::log("Iniciando sistema");
    return 0;
}
```

23. headers .hpp y source .cpp

ESTRUCTURA & GENERICOS

Patrón de organización de proyectos C++. Los archivos de cabecera (`.hpp` o `.h`) contienen las declaraciones e interfaces. Los archivos fuente (`.cpp`) contienen la implementación concreta de dichas funciones y métodos.

```
// === MiClase.hpp ===
#pragma once
class MiClase {
public:
    void ejecutar();
};

// === MiClase.cpp ===
#include <iostream>

void MiClase::ejecutar() {
    std::cout << "Ejecutando método...\n";
}
```

24. #include

ESTRUCTURA & GENERICOS

Directiva del preprocesador que inserta el contenido completo de un archivo de cabecera. Se usan paréntesis angulares (`<vector>`) para librerías estándar o de sistema, y comillas (`"mi_header.hpp"`) para archivos del proyecto.

```
#include <iostream> // Librería Estándar
#include "MiClase.hpp" // Header local del proyecto

int main() {
    std::cout << "Inclusión correcta.\n";
    return 0;
}
```

25. Templates Básicos

ESTRUCTURA & GENERICOS

Mecanismo de programación genérica en C++. Permite escribir funciones o clases parametrizadas por tipos de datos (`template <typename T>`), permitiendo al compilador generar código específico según el tipo usado.

```
#include <iostream>

template <typename T>
T obtener_maximo(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    std::cout << "Max int: " << obtener_maximo(10, 20) << "\n";
    std::cout << "Max double: " << obtener_maximo(3.14, 2.71) << "\n";
    return 0;
}
```

26. auto

FUNDAMENTOS

Inferencia automática de tipos (C++11). Permite que el compilador deduzca automáticamente el tipo de una variable basándose en la expresión de inicialización, simplificando tipos complejos como iteradores.

```
#include <iostream>
#include <vector>

int main() {
    auto numero = 42; // Deducido como int
    auto pi = 3.14159; // Deducido como double

    std::vector<int> vec = {1, 2, 3};
    auto it = vec.begin(); // Deducido como std::vector<int>::iterator

    std::cout << *it << "\n";
    return 0;
}
```

27. range-based for

FUNDAMENTOS

Sintaxis simplificada introducida en C++11 para iterar sobre todos los elementos de un contenedor o arreglo. Se combina habitualmente con `const auto&` para evitar copias inútiles.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<std::string> frutas = {"Manzana", "Pera", "Uva"};

    for (const auto& fruta : frutas) {
        std::cout << fruta << "\n";
    }
    return 0;
}
```

28. Lambdas Básicas

ESTRUCTURA & GENERICOS

Funciones anónimas e in situ (C++11). Sintaxis: `[captura](parametros) { cuerpo }`. El bloque de captura `[]` define qué variables del entorno circundante están disponibles dentro de la lambda.

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    auto multiplicar = [](int a, int b) { return a * b; };
    std::cout << "5 * 6 = " << multiplicar(5, 6) << "\n";

    std::vector<int> nums = {4, 1, 8, 2};
    std::sort(nums.begin(), nums.end(), [](int a, int b) { return a > b; });
    return 0;
}
```

29. std::optional

STL & CONTENEDORES

Wrapper seguro introducido en C++17 (`<optional>`) que representa un valor que puede o no estar presente. Elimina la necesidad de utilizar valores centinela (ej. -1) o punteros nulos.

```
#include <iostream>
#include <optional>

std::optional<int> buscar_par(int val) {
    if (val % 2 == 0) return val;
    return std::nullopt; // Indica ausencia de valor
}

int main() {
    auto res = buscar_par(4);
    if (res.has_value()) {
        std::cout << "Encontrado: " << res.value() << "\n";
    }
    return 0;
}
```

30. std::variant

STL & CONTENEDORES

Unión tipo-segura fuertemente tipada (C++17, `<variant>`). Almacena un valor de una lista fija de tipos explícitos y conoce en todo momento qué tipo contiene actualmente (a diferencia de las uniones C).

```
#include <iostream>
#include <variant>

int main() {
    std::variant<int, std::string> dato;
    dato = 100; // Almacena int
    dato = "Hola Variant"; // Ahora almacena string

    if (std::holds_alternative<std::string>(dato)) {
        std::cout << std::get<std::string>(dato) << "\n";
    }
    return 0;
}
```

31. Stack vs Heap

MEMORIA & PUNTEROS

Dos regiones de memoria en C++. El **Stack** (pila) almacena datos locales de tamaño fijo de forma ultrarrápida y con liberación automática al salir del ámbito. El **Heap** (montón) almacena memoria dinámica solicitada en tiempo de ejecución (`new` / STL), con mayor tamaño pero gestión explícita.

```
#include <iostream>

int main() {
    int en_stack = 10; // Asignación ultra rápida en el Stack
    int* en_heap = new int(20); // Asignación explícita en el Heap

    delete en_heap; // Liberación obligatoria para Heap
    return 0;
}
```

32. Lifetime de Objetos

MEMORIA & PUNTEROS

Intervalo continuo durante la ejecución desde que se completa la construcción de un objeto hasta que inicia su destrucción. Acceder a un objeto fuera de su *lifetime* produce Comportamiento Indefinido (UB).

```
#include <iostream>

struct Tester {
    Tester() { std::cout << "Nace\n"; }
    ~Tester() { std::cout << "Muere\n"; }
};

int main() {
    std::cout << "Inicio\n";
    {
        Tester t; // Inicia Lifetime
    } // Finaliza Lifetime
    std::cout << "Fin\n";
    return 0;
}
```

33. Ownership (Propiedad de Recursos)

MEMORIA & PUNTEROS

Concepto que dicta qué entidad (objeto, función o smart pointer) es responsable de gestionar el ciclo de vida y la posterior liberación de un recurso (memoria, descriptores de archivo, sockets).

```
#include <iostream>
#include <memory>

class Recurso {};

int main() {
    // std::unique_ptr posee exclusivamente el recurso
    std::unique_ptr<Recurso> dueño1 = std::make_unique<Recurso>();
    // std::unique_ptr<Recurso> dueño2 = dueño1; // ERROR: propiedad única
    return 0;
}
```

34. RAII

MEMORIA & PUNTEROS

Resource Acquisition Is Initialization. Idioma fundamental de C++: la adquisición de recursos se realiza en el constructor de una clase y la liberación garantizada en el destructor, logrando gestión automática de memoria y excepciones.

```
#include <iostream>
#include <fstream>

void escribir_log() {
    std::ofstream archivo("log.txt"); // Abre recurso en constructor
    archivo << "Mensaje de prueba";
} // Cierra el archivo automáticamente en el destructor de std::ofstream

int main() {
    escribir_log();
    return 0;
}
```

35. new / delete

MEMORIA & PUNTEROS

Operadores para reserva y liberación manual de memoria dinámica en el Heap. Si una reserva con **new** no se acompaña de su correspondiente **delete** (o **delete[]** para arreglos), se produce una fuga de memoria (memory leak). En C++ moderno se prefieren smart pointers.

```
#include <iostream>

int main() {
    int* ptr = new int(50); // Reserva en el Heap
    std::cout << *ptr << "\n";
    delete ptr; // Liberación explícita
    ptr = nullptr; // Evita puntero colgante
    return 0;
}
```

36. `std::unique_ptr`

SMART POINTERS

Puntero inteligente de propiedad exclusiva (C++11, `<memory>`). Garantiza que solo exista un dueño del puntero subyacente. Libera automáticamente la memoria al destruirse y no puede copiarse, solo transferirse mediante movimiento (`std::move`).

```
#include <iostream>
#include <memory>

int main() {
    auto uptr = std::make_unique<int>(100);
    std::cout << *uptr << "\n";

    auto uptr2 = std::move(uptr); // Transfiere la propiedad
    // uptr ahora es nullptr
    return 0;
}
```

37. `std::shared_ptr`

SMART POINTERS

Puntero inteligente de propiedad compartida (C++11). Mantiene un contador interno de referencias que se incrementa al copiar y decrementa al destruir. La memoria administrada se libera cuando el contador llega a cero.

```
#include <iostream>
#include <memory>

int main() {
    std::shared_ptr<int> s1 = std::make_shared<int>(42);
    {
        std::shared_ptr<int> s2 = s1; // Referencias = 2
        std::cout << "Uso: " << s1.use_count() << "\n";
    } // s2 sale del scope -> Referencias = 1
    std::cout << "Uso: " << s1.use_count() << "\n";
    return 0;
}
```

38. `std::weak_ptr`

SMART POINTERS

Puntero inteligente no propietario que observa a un `std::shared_ptr` sin incrementar el contador de referencias. Resuelve ciclos de referencias circulares. Se convierte a `std::shared_ptr` mediante `.lock()` antes de usarse.

```
#include <iostream>
#include <memory>

int main() {
    std::shared_ptr<int> s = std::make_shared<int>(10);
    std::weak_ptr<int> w = s; // No incrementa ref count

    if (auto locked = w.lock()) { // Convierte a shared_ptr seguro
        std::cout << "Valor: " << *locked << "\n";
    }
    return 0;
}
```

39. Move Semantics

SEMÁNTICA MOVE & REF

Semántica de movimiento (C++11). Permite transferir recursos internos (ej. buffers de memoria) desde un objeto temporal o en desuso (rvalue) hacia un nuevo objeto sin realizar copias profundas, optimizando masivamente el rendimiento.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v1 = {1, 2, 3, 4, 5};
    // Transfiere el buffer de v1 a v2 instantáneamente
    std::vector<int> v2 = std::move(v1);

    std::cout << "v2 size: " << v2.size() << "\n";
    std::cout << "v1 size: " << v1.size() << "\n"; // Queda vacío
    return 0;
}
```

40. std::move

SEMÁNTICA MOVE & REF

Función de utilidad (`<utility>`) que realiza un cast incondicional de una expresión lvalue a un rvalue reference (`T&&`). No mueve nada por sí misma, sino que habilita las funciones de movimiento (Move Constructor / Move Assignment).

```
#include <iostream>
#include <utility>
#include <string>

int main() {
    std::string s1 = "Texto largo";
    std::string s2 = std::move(s1); // Cast a rvalue reference

    std::cout << "s2: " << s2 << "\n";
    return 0;
}
```


41. Copy Constructor

CICLO DE VIDA & REGLAS

Constructor de copia con la firma `T(const T& other)`. Se invoca automáticamente cuando se inicializa un objeto a partir de otro objeto existente del mismo tipo, realizando una copia explícita de sus miembros.

```
#include <iostream>

class Vector2D {
public:
    int x, y;
    Vector2D(int x, int y) : x(x), y(y) {}
    // Copy Constructor
    Vector2D(const Vector2D& other) : x(other.x), y(other.y) {
        std::cout << "Copiado!\n";
    }
};

int main() {
    Vector2D v1(10, 20);
    Vector2D v2 = v1; // Invocación del Copy Constructor
    return 0;
}
```

42. Move Constructor

CICLO DE VIDA & REGLAS

Constructor de movimiento con firma `T(T&& other) noexcept`. Transfiere la propiedad de los recursos de `other` al objeto en creación, dejando a `other` en un estado válido pero vacante.

```
#include <iostream>

class Buffer {
    int* data;
public:
    Buffer(int val) : data(new int(val)) {}
    // Move Constructor
    Buffer(Buffer&& other) noexcept : data(other.data) {
        other.data = nullptr; // Roba el recurso
        std::cout << "Movido!\n";
    }
    ~Buffer() { delete data; }
};

int main() {
    Buffer b1(100);
    Buffer b2 = std::move(b1); // Invoca Move Constructor
    return 0;
}
```

43. Copy Assignment Operator

CICLO DE VIDA & REGLAS

Operador de asignación por copia con firma `T& operator=(const T& other)`. Sobrescribe el contenido de un objeto ya existente copiando los datos de otro objeto.

```
#include <iostream>

class Dato {
public:
    int val;
    Dato& operator=(const Dato& other) {
        if (this != &other) { // Protección de auto-asignación
            val = other.val;
        }
        return *this;
    }
};

int main() {
    Dato d1{10}, d2{20};
    d2 = d1; // Copy Assignment
    return 0;
}
```

44. Move Assignment Operator

CICLO DE VIDA & REGLAS

Operador de asignación por movimiento con firma `T& operator=(T&& other) noexcept`. Libera el recurso actual del objeto destino y le asigna el recurso robado del objeto origen.

```
#include <iostream>

class Caja {
    int* ptr = nullptr;
public:
    Caja(int v) : ptr(new int(v)) {}
    ~Caja() { delete ptr; }

    Caja& operator=(Caja&& other) noexcept {
        if (this != &other) {
            delete ptr;
            ptr = other.ptr;
            other.ptr = nullptr;
        }
        return *this;
    }
};

int main() {
    Caja c1(10), c2(20);
    c2 = std::move(c1); // Move Assignment
    return 0;
}
```

45. Rule of 0

CICLO DE VIDA & REGLAS

Regla de diseño que establece que si tu clase no maneja directamente recursos crudos (como punteros con `new` o desambiguaciones de sistema), no debes declarar ninguno de los 5 métodos especiales. Deja que la STL (`std::vector`, `std::string`) gestione todo.

```
#include <iostream>
#include <string>

class Usuario { // Sigue la Rule of 0
    std::string nombre;
    int id;
    // Compilador genera los 5 métodos especiales óptimos
};

int main() {
    Usuario u1;
    Usuario u2 = u1; // Copia limpia automática
    return 0;
}
```

46. Rule of 5

CICLO DE VIDA & REGLAS

Principio de C++11: Si una clase requiere definir explícitamente cualquiera de los siguientes métodos (usualmente por gestionar un recurso crudo), debe definir los cinco: **Destructor**, **Copy Constructor**, **Move Constructor**, **Copy Assignment** y **Move Assignment**.

```
#include <iostream>

class Recurso {
    int* ptr;
public:
    Recurso() : ptr(new int(0)) {}
    ~Recurso() { delete ptr; } // Destructor
    Recurso(const Recurso& o) : ptr(new int(*o.ptr)) {} // Copy Const
    Recurso(Recurso&& o) noexcept : ptr(o.ptr) { o.ptr = nullptr; } // Move Const
    Recurso& operator=(const Recurso& o) { if(this!=&o){ *ptr = *o.ptr; } return *this; } // Copy
    Recurso& operator=(Recurso&& o) noexcept { if(this!=&o){ delete ptr; ptr=o.ptr; o.ptr=nullptr; }
    return *this; } // Move Assign
};

int main() {
    Recurso r1;
    Recurso r2 = std::move(r1);
    return 0;
}
```

47. Referencias const

SEMÁNTICA MOVE & REF

Referencia de solo lectura (**const T&**). Permite recibir argumentos eficientemente sin realizar copia y aceptando tanto variables (lvalues) como temporales (rvalues). Es la forma predilecta de pasar parámetros leídos en C++.

```
#include <iostream>
#include <string>

void imprimir_largo(const std::string& str) {
    std::cout << str << " (Longitud: " << str.length() << ")\n";
}

int main() {
    std::string s = "Variable";
    imprimir_largo(s); // Acepta lvalue
    imprimir_largo("Literal C++"); // Acepta rvalue temporal
    return 0;
}
```

48. Referencias Rvalue &&

SEMÁNTICA MOVE & REF

Referencia a objetos temporales o expirantes (**T&&**). Introducida en C++11, permite identificar valores rvalue para robar sus recursos en constructores u operadores de movimiento, habilitando semántica de movimiento eficiente.

```
#include <iostream>

void procesar(int& x) { std::cout << "Lvalue ref: " << x << "\n"; }
void procesar(int&& x) { std::cout << "Rvalue ref: " << x << "\n"; }

int main() {
    int a = 10;
    procesar(a); // Invoca versión lvalue
    procesar(20); // Invoca versión rvalue (temporal)
    return 0;
}
```